TP – Vérification d’e-mail avec messagerie (Spring Boot + RabbitMQ)

Durée : 1 à 2 séances (3h + 3h)

Niveau : M1 Informatique – Architecture & Services

1. Objectifs pédagogiques

- Comprendre le découplage entre services via la messagerie (RabbitMQ).
- Mettre en place un flux d’inscription avec envoi d’e-mail asynchrone et vérification par lien.
- Implémenter la génération et la validation d’un token de vérification stocké sous forme de hash.
- Configurer un échange, une file et une DLQ RabbitMQ.
- Manipuler Spring Boot (Web, Data JPA, AMQP) et un serveur SMTP local (MailHog).

2. Contexte fonctionnel

Vous construisez un mini-système composé de deux microservices : Auth (inscription & vérification) et Notification (envoi d’e-mails). À l’inscription, Auth publie un événement "UserRegistered" dans RabbitMQ. Notification consomme cet événement et envoie un e-mail contenant un lien de vérification. L’utilisateur clique ce lien ; Auth valide alors le token et marque le compte comme vérifié. Optionnellement, Auth peut publier "EmailVerified" pour l’analytics.

3. Architecture cible (vue simple)

Client → Auth (POST /register) → publie sur exchange "auth.events" → Queue "notification.user-registered" → Notification envoie e-mail (SMTP MailHog). Lien de vérification (GET /verify) côté Auth. (Optionnel) Auth publie "EmailVerified".

4. Pourquoi un hash pour le token ?

Principe — Le token de vérification est un secret à usage unique. Comme pour les mots de passe, **on ne stocke jamais le token en clair en base** : on stocke uniquement un **hash** (ex. BCrypt). Lors de la vérification, le token reçu est hashé et comparé au hash stocké. Ainsi, une fuite de base ne permet pas d’utiliser les tokens.

Bénéfices — Confidentialité (pas de secret en clair), intégrité (token non modifiable), révocation simplifiée (suppression de l’entrée), conformité (réduction des données sensibles stockées).

5. Pourquoi passer par une messagerie ?

Découplage — Auth n'attend pas la fin de l'envoi d'e-mail ; il publie un fait métier. Le service Notification peut évoluer et scaler indépendamment.

Résilience & scalabilité — En cas de pic d'inscriptions ou d'indisponibilité temporaire SMTP, les messages s'accumulent dans la file et sont traités dès que possible (retries, DLQ).

Traçabilité — Les événements structurés peuvent être audités (eventId, timestamps, correlationId).

6. Rôles des composants / services

Service Auth — Expose "POST /register" et "GET /verify" ; gère la persistance des utilisateurs et des tokens (stockage du hash), publie "UserRegistered" et (optionnel) "EmailVerified".

Service Notification — Consomme "UserRegistered", construit le lien de vérification et envoie l'e-mail via SMTP ; gère les erreurs (retry, DLQ).

RabbitMQ — Fournit l'échange (topic), la file de consommation et la file de retombée (DLQ).

MailHog — Serveur SMTP de développement et boîte de réception web locale pour tester l'e-mail sans envoyer à l'extérieur.

7. Spécifications des événements

UserRegistered { eventId, userId, email, tokenId, tokenClear?, occurredAt } (le champ tokenClear est inclus uniquement dans la version la plus simple du TP).

EmailVerified (optionnel) { eventId, userId, occurredAt }

En-têtes recommandés : x-correlation-id, x-schema-version.

8. Contrats d'API (Auth)

POST /register — body: { email, password? } → 201 Created ; effets: crée l'utilisateur (verified=false), crée un token (hash stocké), publie UserRegistered.

GET /verify?tokenId=...&t=... — 200 si vérifié, 400 si invalide/expiré ; effets: set verified=true, supprime le token (one-shot).

9. Modèle de données (simplifié)

User(id, email, verified:boolean)
VerificationToken(tokenId, userId, tokenHash, expiresAt)

10. Informations de configuration

Docker Compose — RabbitMQ (ports 5672/15672), MailHog (1025/8025).

Spring – Auth & Notification — Dépendances: spring-boot-starter-web, spring-boot-starter-amqp, (Auth) spring-boot-starter-data-jpa + h2, (Notification) spring-boot-starter-mail.

RabbitMQ (application.yml) — exchange: auth.events ; routing keys: auth.user-registered, auth.email-verified ; déclarer une DLX (auth.events.dlq) et des DLQ par file.

SMTP (Notification) — host=localhost, port=1025 (MailHog), sans TLS ni auth en local.

11. Étapes à réaliser – Séance 1

1. Implémenter POST /register dans Auth : persister l'utilisateur (verified=false).
2. Générer un token (UUID), stocker BCrypt(token) et expiresAt en base ; conserver tokenId.
3. Publier l'événement UserRegistered sur l'échange auth.events avec la routing key auth.user-registered.
4. Dans Notification, déclarer l'échange, la file notification.user-registered et la DLQ ; consommer l'événement.
5. Construire le lien http://localhost:8080/verify?tokenId=...&t=... et envoyer l'e-mail via MailHog.
6. Tester : POST /register → vérifier l'e-mail reçu dans http://localhost:8025.
7. Implémenter GET /verify côté Auth : vérifier expiration, comparer BCrypt(t) au tokenHash, marquer verified=true et supprimer le token (one-shot).

12. Étapes à réaliser – Futur

8. Rendre l'opération idempotente (ne pas échouer si le compte est déjà vérifié).
9. Configurer la DLQ et simuler une erreur côté Notification pour observer le message en DLQ.
10. publier EmailVerified et afficher un compteur dans un consumer Analytics minimal.

13. Critères d'évaluation

- Flux fonctionnel complet : inscription → e-mail → vérification OK (40%).
- Sécurité minimale : token non stocké en clair, expiration respectée (25%).
- Messagerie : configuration correcte, DLQ opérationnelle (20%).
- Qualité : logs clairs, README succinct, tests manuels reproductibles (15%).

14. Conseils & bonnes pratiques

Minimiser les PII dans les événements (éviter les données inutiles). Utiliser des identifiants stables (userId) comme clés de routage. Ajouter eventId (UUID) pour ignorer les doublons côté consommateurs. Définir un TTL pour les tokens (15–30 min). Journaliser les correlationId pour suivre une inscription de bout en bout.

Annexe A — Exemples de configuration (YAML)

```yaml
# application.yml (extraits)
spring:
  rabbitmq:
    host: localhost
    port: 5672
    username: guest
    password: guest
app:
  mq:
    exchange: auth.events
    rk:
      userRegistered: auth.user-registered
      emailVerified: auth.email-verified
---
# Notification – SMTP vers MailHog
spring:
  mail:
    host: localhost
    port: 1025
    properties:
      mail.smtp.auth: false
      mail.smtp.starttls.enable: false
```

Annexe B — Schéma des événements (JSON)

```json
{
  "type": "UserRegistered",
  "eventId": "uuid",
  "occurredAt": "2026-02-15T10:00:00Z",
  "data": {
    "userId": "u_123",
    "email": "a***@ex.com",
    "tokenId": "tok_abc",
    "tokenClear": "OPTIONNEL (flux simple TP)"
  },
```

```
  "headers": {
    "x-correlation-id": "uuid",
    "x-schema-version": 1
  }
}
```